Department of Electronic & Telecommunication Engineering
University of Moratuwa

EN3021 - Digital System Design

Pipelined Five Stage CPU Design

Group x86

Chandira R.M.	200082E
Fernando K.N.A.L.	200164H
Haputhanthri H.H.A.M.	200207U

31 December 2023.

Contents

1 Introduction **2**

2 Architecture diagram **2**

3 Design decisions and Justifications **2**

3.1 Data Forwarding Unit 2

3.2 Two Bit Branch Prediction 3

3.3 Set Associative Cache 3

4 Utilization report **3**

5 Performance **3**

6 Contribution **4**

7 Reference **5**

1 Introduction

This project requires to develop a pipelined five stage CPU design based on RV32I instruction set. The processor consist of 3 bus structure with a micro programmed controller.
The processor is designed to execute all R type, I type, S type and SB type instructions of RV32I instruction set.

2 Architecture diagram

The design implementation of the processor is as follows.

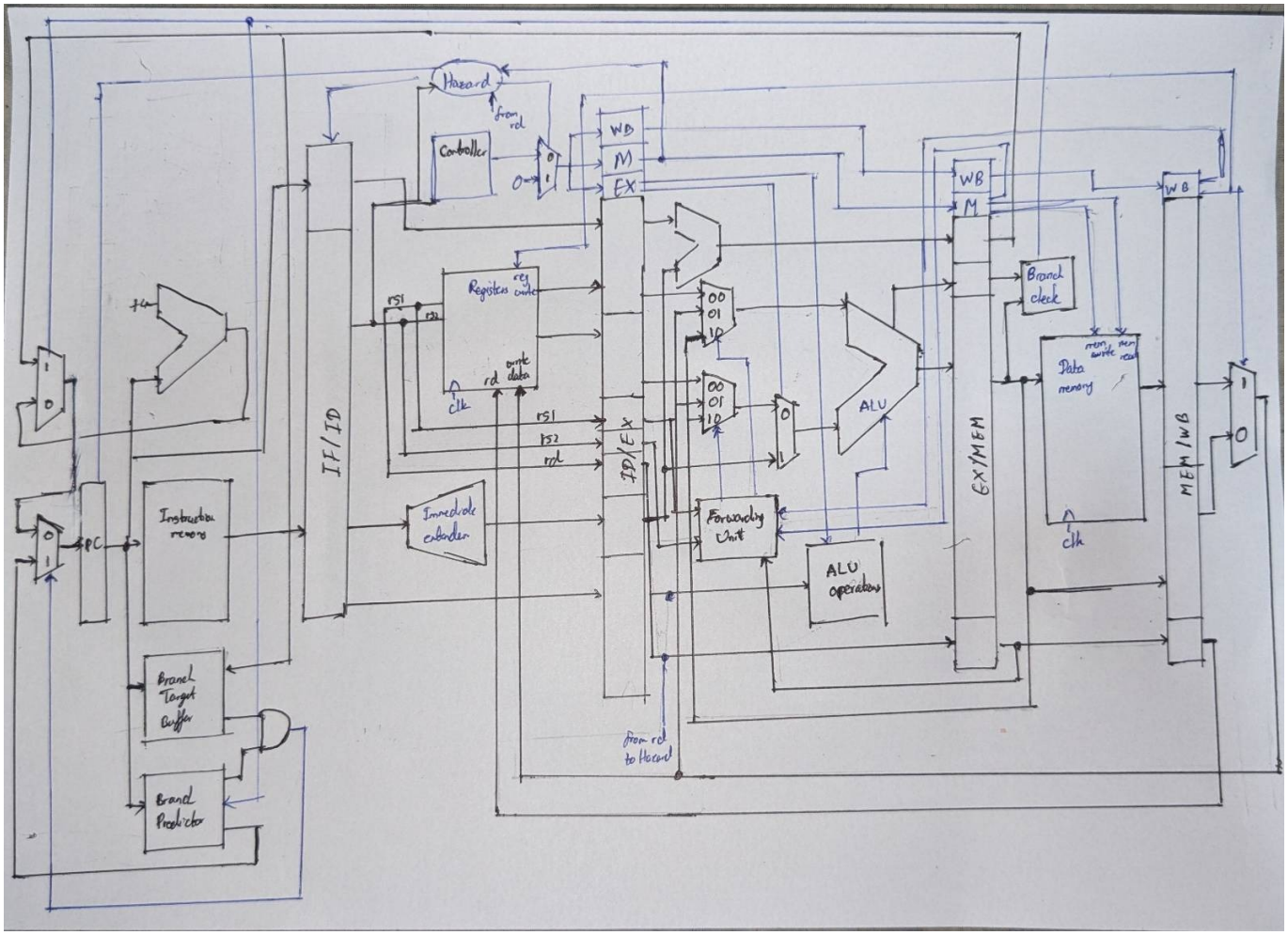

Figure 1: Pipeline architecture

3 Design decisions and Justifications

The following design decisions were taken to increase processor performance and efficiency.

3.1 Data Forwarding Unit

To handle the data hazards of read after write data forwarding unit was implemented. With this implementation the need to stall the processor for 1 cycle for the aforementioned data hazard is eliminated, respectively.

3.2 Two Bit Branch Prediction

A two bit branch predictor was implemented alongside the branch target buffer to predict the branch targets. Using a branch predictor reduce the need to stall the processor operations for 3 cycles by identifying the target instruction address in **IF** *Instruction Fetch* stage. Using two bit branch prediction increase the efficiency of the branch prediction than a single bit branch predictor in the case of high regular branches like loops.

3.3 Set Associative Cache

Independent caches were implemented for data memory and instruction memory. Size of each main memory is 128 bytes, byte addressable memory while size of each cache memory is 32 bytes.

Each cache has two sets, *set 0* and *set 1*. Each set holds 16 bytes and stores two adjacent bytes from the main memory upon a requested read operation. Caches use FIFO *First In First Out* replacement algorithm is implemented by a **Flag_bit** for each set. A three bit wide **Tag_bits** are used for each two word width row in the cache to store the remaining address length. A **Dirty_bit** is assigned to each row to show a change in main memory which is yet to be reflected in the cache.

The size of the cache is based on the size of main memory. The size of the sets was chosen to have high search speeds in tag bits to increase performance. FIFO replacement algorithm was chosen for simplicity in design.

4 Utilization report

The resources used for the design post implementation is as follows.

Table 1: Utilization report

Resource	**Utilization**	**Available**	**Utilization %**
LUT	1116	17600	6.34
FF	1342	35200	3.81
DSP	3	80	3.75
IO	8	100	8.00
BUFG	7	32	21.88

- LUT : Look up table, typically with six inputs.
- LUTRAM : Look up table Random Access Memory, similar to LUTs but with added capability of storing data.
- DSP : Digital Signal Processing IP
- IO : Input / Output pins
- BUFG : Buffer clock global, used to distribute clock signals around FPGA

5 Performance

The design is capable of running at 50Mhz while meeting timing requirements.

6 Contribution

Member	**Contribution**
Chandira R.M.	Documentation, Set Associative Cache Implementation
Fernando K.N.A.L.	Pipeline Implementation, Data Forwarding Unit Implementation
Haputhanthri H.H.A.M.	Branch Prediction Implementation , Implementation on FPGA

7 Reference

[1] Zwolinski Dr Mark, Digital System Design With Systemverilog. Pearson Education India, 2010.

[2] S. Sutherland and D. Mills, "Synthesizing SystemVerilog Busting the Myth that SystemVerilog is only for Verification." Accessed: Oct. 15, 2023. [Online]. Available: https://sutherland-hdl.com/papers/2013-SNUG-SV_Synthesizable-SystemVerilog_paper.pdf

[3] D. A. Patterson and J. L. Hennessy, Computer Organization And Design The Hardware/Software Interface. Cambridge, Ma Morgan Kaufman Publishers, 2018.

[4] "ChipVerify," ChipVerify. https://www.chipverify.com/ (accessed Oct. 15, 2023).

[5] A. Waterman and K. Asanović, "The RISC-V Instruction Set Manual Volume I: User-Level ISA Document Version 2.2," 2017. Available: https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf

[6] "RV32I, RV64I Instructions — riscv-isa-pages documentation," msyksphinz-self.github.io. https://msyksphinz-self.github.io/riscv-isadoc/html/rvi.html#slt

[7] "Zybo Reference Manual - Digilent Reference," digilent.com. https://digilent.com/reference/programmable-logic/zybo/reference-manual